8

Introduction to protocols

A protocol can be defined as a set of rules governing the exchange of data between a transmitter and a receiver over a communications link or network.

Objectives

When you have completed studying this chapter you will be able to:

- Define the term ‘protocol’
- Describe the role of protocols in flow control
- Describe the two most popular types of flow control protocols:
  - XON/XOFF
  - ETX/ACK
- Differentiate between and describe the two modes of binary synchronous protocols:
  - Point-to-point
  - Multipoint
- Describe HDLC and SDLC protocols in terms of:
  - Frame format
  - Frame contents
  - Operation
  - Error/flow control
- Describe file transfer protocols
- Describe ARQ protocols

As we will see in Chapter 9, data communications systems, which follow the OSI reference model, are made up of several hierarchical layers. Each of these layers contains working software or hardware elements referred to as entities. One of the elements in each layer is a protocol entity, which has its own specification, and broadly speaking is a protocol in its own right. The purpose of the protocol entity is to determine how messages are transferred across a network to a peer entity in another node.

Actual transfers across the physical link are determined by the data link layer protocol, and it is these types of protocols that are discussed in this chapter. Other more complex protocols are discussed in later chapters.

A protocol is concerned with some or all of the following:

- Initialization, to start the transmission of data.
- Framing and frame synchronization – this defines the beginning and end of a frame and ensures that the receiver can synchronize with the frame.
- Flow control, to ensure that the receiver is not swamped.
- Line control, applies to half duplex links in which the transmitter tells the receiver to start transmitting.
- Error control.
- Timeout control, so action can be taken if no acknowledgment is received within a certain period.

8.1 Flow control protocols

The most elementary protocols are only concerned with flow control and were introduced as an improvement on simple techniques such as the insertion of delays between characters or echoing of received characters to the transmitter. The two most popular types of flow control protocols are XON/XOFF and ETX/ACK.

8.2 XON/OFF

This is a character based flow-control protocol, which uses two special characters. Typically, these are the ASCII characters DC1 for XON and DC3 for XOFF. The transmitter sends data until it receives an XOFF from the receiver; it then waits for an XON before resuming transmission. A typical example can be found in a printer buffer. When the buffer reaches a certain point (say 66%), the printer sends an XOFF to the PC, then sends an XON when the buffer is emptied to another point (say 33%).

One disadvantage of XON/XOFF is that the data stream being sent may contain one of the control characters, although this is not a problem in applications such as printer control.

8.3 Binary synchronous protocol

The binary synchronous control (BSC) protocol was designed by IBM in 1966 for computer-to-terminal and computer-to-computer communications. It can be used in point-to-point or in multipoint mode. BSC is a character-based protocol as opposed to the high-level data link control (HDLC) protocol, which is bit-based. HDLC is discussed under ‘HDLC and SDLC protocols’ below.

The XON/XOFF flow control mechanism can easily handle short interactive messages between a terminal and a computer. They are less adequate in ‘block mode’, that is when passing complete messages, with hundreds or even thousands of characters, between terminals. The BSC protocol, on the other hand, was specifically designed to handle large blocks of data.

Control characters are used to separate the different fields in a BSC message and for the exchange of acknowledgment information.

Table 8.1 lists the control characters involved in BSC.

Abbreviation	Meaning	Description
ACK	Acknowledgment	Received block is OK
ACK1	Acknowledgment 1	Received odd block is OK
ACK2	Acknowledgment 2	Received even block is OK
DLE	Data Link Escape	Control character follows
ENQ	Enquiry	Please respond with data
EOT	End of Transmitted Data	The transmission is over
ETB	End of Transmission Block	End of block
ETX	End of Text	End of data text
ITB	End of Intermediate Block	More blocks coming
NAK	No Acknowledgment	Problems with received blocks
SOH	Start of Header	Routing information
STX	Start of Text	Message data text commences
SYN	Synchronous Idle	Allows receiver to synchronize

Table 8.1
BSC control characters

There are various types of BSC messages, as given in Figure 8.1.

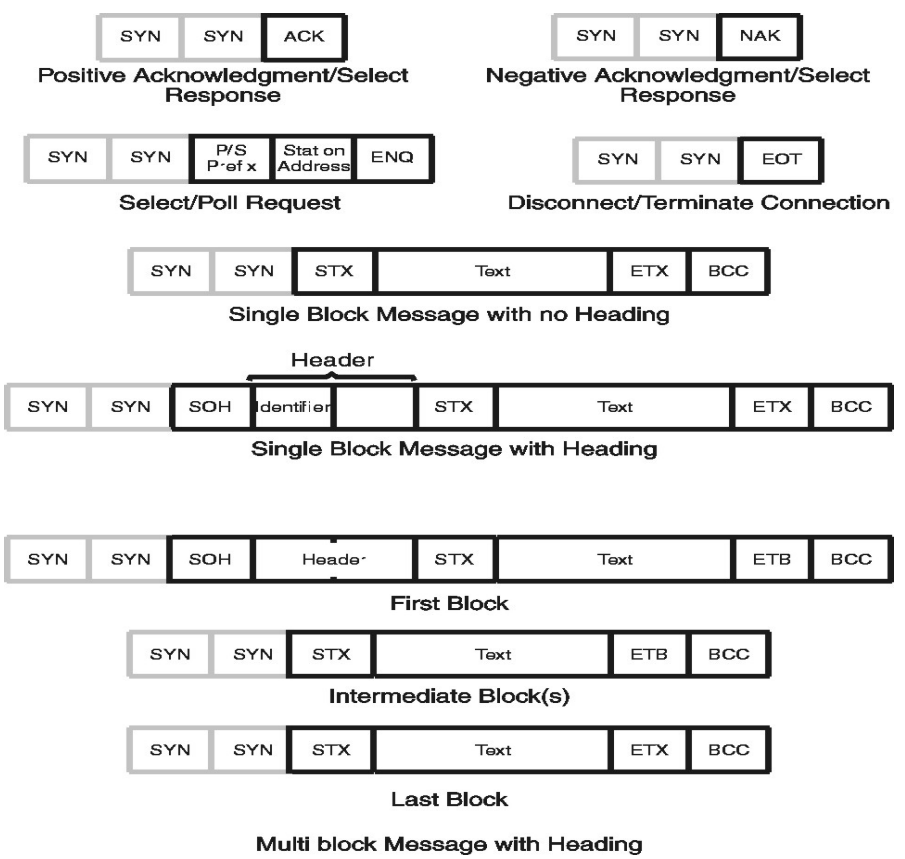


Figure 8.1
BSC message format

The receiver of the message uses the two SYN characters, (bit pattern 0010110) to synchronize with the start of the message. Note that the SYN characters are not considered to be part of the message itself and are therefore not used in the calculation of the block check character (BCC). In order to maintain synchronization, the transmitter inserts SYN characters into text messages once every second; however, as before, these are not used in BCC calculations.

A text field starts with the STX character and ends with ETX, ETB, EOT or ITB, as appropriate. The BCC field consists of a vertical/longitudinal check or a cyclic redundancy check such as the CRC-16 used for transparent mode (see Data Transparency later in this chapter).

If a message is received without error, the receiver responds with ACK1 for the first response, ACK2 for the next, and so on, alternating at each response so that odd numbered messages return ACK1 and even numbered messages return ACK2. This enables the transmitter to track the responses and detect any messages not acknowledged. ACK1 is represented by the sequence DLE00, and ACK2 is represented by the sequence DLE01.

If the receiver detects an error in a message, it responds with a NAK.

Point-to-point mode full sequence transmission is as follows:

Station 1	Station 2	
SYN SYN *		
	Three possible responses:	
	*	Ready to receive SYN SYN DLE 00
	*	Not ready to receive SYN SYN NAK
	*	Try again later SYN SYN NAK
SYN SYN STX [data chars] ETX BCC		
	*	Positive ackn/ACK0 SYN SYN DLE 00
Station 1 terminates the exchange * SYN SYN EOT .		

Table 8.2
Sequence of transmission in point-to-point mode

Multipoint mode

In this mode there is one primary station and one or more secondary stations on the same line.

All exchanges are initiated by the primary station as one of two types of transaction:

- The primary station issues a poll to determine if a secondary station has any data to transmit.
- The primary station selects a secondary station in order to transmit data to it.

Time out functions

- The transmitting station times out if there is no response within three seconds.

- A station on a switched network (the normal telephone system) disconnects itself if there is no activity for 20 seconds.
- A transmitter receiving a TTD or WAK character waits for two seconds before retrying.

Data transparency

- As an ASCII based protocol, BSC cannot usually operate transparently. That is, it is unable to handle binary data because the 7-bit data field limits the range of numbers to 0 to 127. Binary (or hexadecimal) data needs the range 0 to 225 (8 bits).
- It is possible to make BSC handle binary data by using eight data bits (and thus no parity bit) and preceding each BSC control character with a DLE. Thus ETX becomes DLE ETX, and so forth. In this way, control character sequences in the binary data will not be erroneously interpreted as BSC control characters. To avoid the possibility of problems with a DLE in the binary data, the transmitter inserts an additional DLE into the data when it detects a DLE. This is removed by the receiver before it passes on the data.
- Error checking uses a cyclic redundancy check (CRC) polynomial code in transparent operation because the eighth bit is not available for the parity calculation.

BSC limitations

- BSC is a half duplex protocol in which each message must be acknowledged by the receiver. This is very slow compared to more efficient protocols that number each message and send them out in multiples, requiring only an acknowledgment for the group. In transparent mode, the extra number of DLE characters that may be required is wasteful.

8.4 HDLC and SDLC protocols

HDLC has been defined by the International Standards Organization for use on both multipoint and point-to-point links. Other descriptions of it include SDLC (synchronous data link control used by IBM) and ADCCP (advanced data communication control procedure used by ANSI). HDLC will be the reference used throughout the following text. In contrast to the BSC protocol, HDLC is a bit-based protocol. It is interesting to note that it is a predecessor to the local area network datalink protocols.

The two most common modes of operation for HDLC are:

- Unbalanced normal response mode (NRM). This is used with only one primary (or master) station initiating all transactions.
- Asynchronous balanced mode (ABM). In this mode each node has equal status and can act as either a primary or secondary node.

Frame format

The standard format is indicated Figure 8.2. The three different classes of frames used are as follows:

Unnumbered frames: Used for setting up the link or connection and to define whether NRM or ABM is to be used. They are called

unnumbered frames because no sequence numbers are included.

Information frames: Used to convey the actual data from one node to another.

Supervisory frames: Used for flow control and error control purposes. They indicate whether the secondary station is available to receive the information frames; they are also used to acknowledge the frames. There are two forms of error control used: a selective retransmission procedure because of an error, or a request to transmit a number of previous frames.

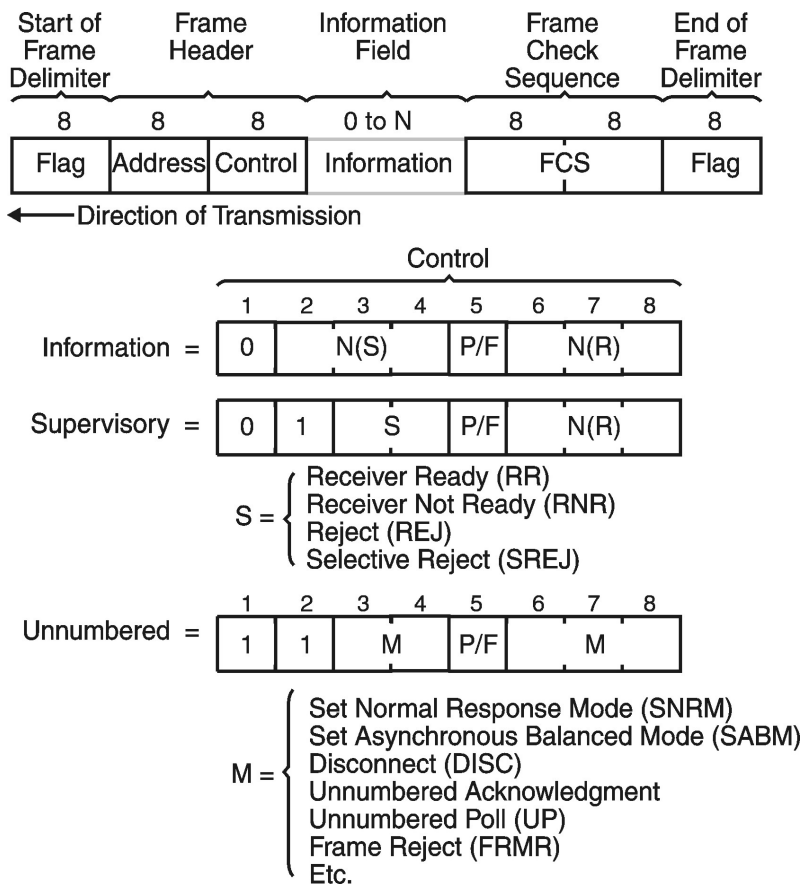


Figure 8.2
HDLC frame format and types

Frame contents

The frame contents are as follows:

- The flag character is a byte with the value 01111110. To ensure that the receiver always knows that the character it receives is unique (rather than merely some other character in the sequence); a procedure called zero insertion is adopted. This requires the transmitter to insert a 0 after a sequence of five 1s in the text, so that the flag character can never appear in the message text. The receiver removes the inserted zeros.

- The frame check sequence (FCS) uses the CRC-CCITT methodology, with sixteen 1s to the tail of the message before the CRC calculation proceeds, and the remainder is inverted.
- The address field can contain one of three types of address for the request or response messages to or from the secondary node:
  - Standard secondary address
  - Group addresses for groups of nodes on the network
  - Broadcast addresses for all nodes on the network (here the address contains all 1s)
- Where there are a large number of secondaries on the network, the address field can be extended beyond eight bits by encoding the least significant bit as a 1. This then indicates that there is another byte to follow in the address field.
- The control field is indicated in Figure 8.2.

Note: The send and receive sequence numbers are important for the detection and correction of errors in messages. The P/F bit is the poll/final bit and when set to 1 indicates to the receiver that it must respond or acknowledge this frame (again with the P/F bit set to 1).

Protocol operation

A typical sequence of operations for a multidrop link is given below:

1 The primary node sends a Normal Response Mode frame, with the P/F bit set to 1, together with the address of the secondary node.
2 The secondary node responds with an unnumbered acknowledgment with the P/F bit set to 1. If the receiving node is unable to accept the setup command, a disconnected mode frame is returned instead.
3 Data is transferred with the information frames.
4 The primary node sends an unnumbered frame containing a disconnect in the control field.
5 The secondary node responds with an unnumbered acknowledgment.

A similar approach is followed for a point-to-point link using asynchronous balanced mode, except that both nodes can initiate the setting up of the link and the transfer of information frames, and the clearing of the point-to-point link. The following differences also apply:

- When the secondary node transfers the data, it transmits the data as a sequence of information frames with the P/F bit set to 1 in the final frame of the sequence.
- In NRM mode, if the secondary node has no further data to transfer, it responds with a Receiver Not Ready frame with the P/F bit set to 1.

Error control/flow control

For a half duplex exchange of information frames, error control is by means of sequence numbers. Each end maintains a transmit sequence number and a receive sequence number. When a node successfully receives a frame, it responds with a supervisory frame

containing a receiver ready (RR) indication and a receive sequence number. The number is that of the next frame expected, thus acknowledging all previous frames.

If the receiving node responds with a negative acknowledgment (REJ) frame, the transmitter must transmit all frames from the receive sequence number in the REJ frame. This happens when the receiver detects an out-of-sequence frame.

It is also possible for selective retransmission to be used. In this case the receiver would return a selection rejection frame containing only the sequence number of the missing frame.

A slightly more complex approach is required for a point-to-point link using asynchronous balanced mode with full duplex operation, where information frames are transmitted in two directions at the same time. The same philosophy is followed as for half duplex operation except that checks for correct sequences of frame numbers must be maintained at both ends of the link.

Flow control operates on the principle that the maximum number of information frames awaiting acknowledgment at any time is seven. If seven acknowledgments are outstanding, the transmitting node will suspend transmission until an acknowledgment is received. This can be either in the form of a receiver ready supervisory frame, or piggybacked in an information frame being returned from the receiver.

If the sequence numbers at both ends of the link become so out of sequence that the number of frames awaiting acknowledgment exceeds seven, the secondary node transmits a frame reject or a command reject frame to the primary node. The primary node then sets up the link again, and on an acknowledgment from the secondary node, both sides reset all the sequence numbers and commence the transfer of information frames.

It is possible for the receiver to run out of buffer space to store messages. When this happens it will transmit a receiver not ready (RNR) supervisory frame to the primary node to instruct it to stop sending any more information frames.

8.5 File transfer protocols

In most asynchronous file transfers used on PCs, the basic structure is the packet (or frame) consisting of a group of fields. Only one of these fields contains the actual data. The remaining fields, known as service fields, contain the information required for the receiver to verify that the packet is error free.

Automatic repeat request (ARQ) protocols

The most common type of packet protocol is the automatic repeat request (ARQ) protocol in which an error detected in a received packet and an unacknowledged packet automatically results in the retransmission of the packet.

Send and wait ARQ

Here the receiver inputs the packets and after verifying that the packet is in the correct sequence relative to the previous packet, computes a local check value on the data portion of the packet. On a successful match with the one in the packet the receiver acknowledges with an ACK; or sends a NAK. When the transmitter receives the ACK it then sends the next packet.

Continuous ARQ

Here the transmitter sends several packets in a row with no delay between packets. The receiver sends a NAK or ACK (as per the send and wait ARQ) together with the packet's

number. The transmitter continually examines the stream of acknowledgments returning and keeps track of the packets with errors. At the end of the transmission the packets with errors are retransmitted.

The send and wait ARQ remains the most popular file transfer protocol found on PCs.

Packet design

There are three approaches for design of packets as shown in the figures below.

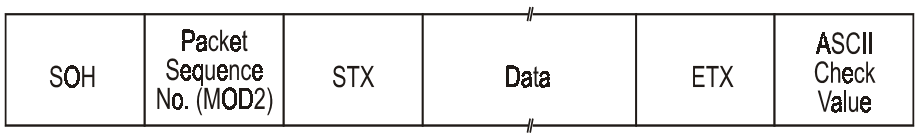


Figure 8.3
Packet for transfer of text files ***(64 to 512 bytes)***

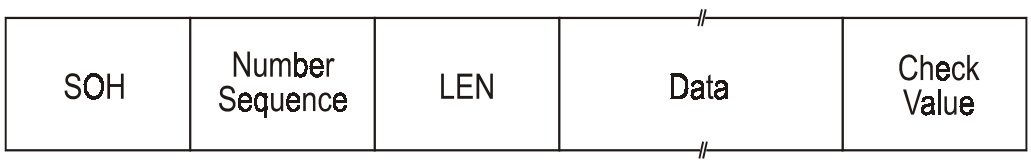


Figure 8.4
Packet for transfer of binary files



Figure 8.5
Packet for fixed length data field packet

The two types of protocols that are used for file transfer are XMODEM and Kermit.

XMODEM

XMODEM is a simple send and wait ARQ protocol that uses a fixed length data field. The check value is a single byte arithmetic checksum.

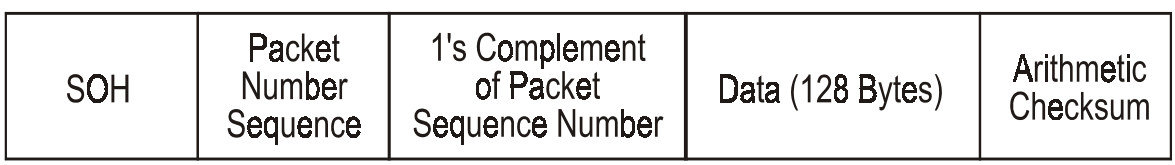


Figure 8.6
Layout of XMODEM protocol

SOH	Start of header byte
Packet sequence No	Current packet number (up to 256) 1's complement of the current packet
Packet sequence	That 2's complement of the current packet number in the previous field
Data	Length of data (binary or text) is fixed at 128 bytes
Arithmetic checksum	A 1 byte arithmetic sum of the data field only – modulo 256

Table 8.3
Bytes in the XMODEM protocol

The mode of operation of the XMODEM protocol is briefly as follows:

- The receiver sends a NAK to transmitter to initiate transmission.
- The transmitter then sends a packetized 128 byte block of data. An ACK received by the transmitter commences transmission of the next packet. A NAK receipt means retransmit the packet; whilst a CAN aborts the transfer.
- When all the data has been sent, a sender transmits a solitary EOT, which the receiver acknowledges with an ACK.

Other versions of XMODEM use a 1 byte CRC instead of the single byte arithmetic checksum (XMODEM-CRC). This is a non-reversed CRC algorithm with the CCITT divisor polynomial $X^{16} + X^{12} + X^5 + 1$.

The problems with XMODEM

- The XMODEM protocol is designed to operate at lower error rates on communication links. Unless an in-built error-correction algorithm is used within the modem the XMODEM protocol will fail. Higher baud rates and lower quality telecommunication services (that provide a cheaper service) make the problem worse.
- Line noise with multiple bit changes often pass through XMODEM's simple error detection system undetected.
- The simple response control characters (ACK/NAK/EOT) are often corrupted to other codes resulting in erroneous actions taken with a resulting waste of time. If the characters are corrupted to the control-X character, file transfer is aborted.
- Sliding windows giving more efficiency in the use of the protocol cannot be implemented, as each NAK or ACK response character does not have an accompanying sequence number. This makes for inefficient operation over communication systems with long time delays.
- 8 bits are required in the construction of the characters in the packet. This makes it inappropriate for communication systems, which only allow 7 bits for each character.

The big advantage with XMODEM is that it is provided on all popular communication packages (such as the Windows Terminal package). This *de facto* standard is useful for transferring files between different incompatible computer systems, which have

XMODEM as the only common denominator. It should be noted however that the XMODEM is often merely then used to transfer the more efficient actual file transfer protocol (such as ZMODEM) from one machine to the other. This is then used to affect the actual file transfer. The result can be a huge saving in time and thus costs of communication time.

YMODEM

This was introduced as an improvement to the XMODEM protocol. This protocol has enjoyed wide spread acceptance as it was released into the public domain and it was written in C.

Features include:

- YMODEM has less overhead than XMODEM in its frame structure with 1024 bytes allowed to be transferred per block. YMODEM still has the flexibility to reduce the data block size to 128 bytes if the error rate is too high (causing repeated retransmissions).
- YMODEM is more reliable than XMODEM. In order to terminate a file transfer, the character sequence: control-X, control-X (or two CAN characters) must be received. This prevents unnecessary file transfer terminations due to errors on the line.
- CRC error detection is used to ensure a high level of error detection.
- File related information (file name; file time; file date; size) is also transferred to the receiving computer.
- Multiple files can be transferred using the batch file capability of the protocol.
- 8 data bits are the basis of file transfer. This causes some problems as discussed earlier with XMODEM.
- The stop and wait technique is used to send data packets. This lack of an adequate sliding window is the cause of inefficiencies on systems with significant time delays.

ZMODEM

This was developed to overcome the limitations in the previous file transfer protocols.

Typical features of the ZMODEM protocols are as follows:

- In addition to batch file transfer, which is a standard feature, a useful feature is that ZMODEM can recommence the file transfer at any point at which the communication link failed (to the precise byte at which communications were lost).
- ZMODEM will not perform a file transfer if the same file already exists on the receiver's hard disk. This check is performed automatically by ZMODEM. This saves on time.
- Better error detection and correction is performed by negotiating the use of the CRC-32 error detection mechanism.
- Data compression can be performed; thus speeding up file transfer.
- The sliding windows technique is used to improve performance over communication links with time delays. This size of the packet can be modified automatically by the software, downwards from 1024 bytes, to handle increasing noise on the communications link.

Kermit

The basic Kermit is a send and wait ARQ packet protocol. The sender transmits a packet and then awaits the receiver's acknowledgment of the packet. The receiver can then either request the next packet (ACK) or retransmission of the previous packet (NAK).

Although there are a number of similarities with XMODEM the main differences are:

- Kermit can transfer a number of files in one file transfer session.
- Packets may be of variable length.
- The I/O channels only have to transfer printable ASCII characters.
- Several types of packets are defined.
- The receiver's responses must consist of entire packets.
- Transmitter and receivers negotiate important operating parameters such as device padding, etc.
- Name of file is included in protocol.
- The herald packets make the protocol extensible.

The operating sequence of Kermit commences in a similar way to XMODEM where the receiver sends repeated NAK packets until the transmitter responds by sending a herald packet, which in Kermit is called a send initiate packet. The receiver makes its own preference known by including them in its ACK packet. When the entire file is transferred the sender transmits a special end of file packet. If there are more files to transmit, it sends the file header packet for the next file. When all files have been sent, the transmitter sends an end of transmission packet to signal the end of the session.

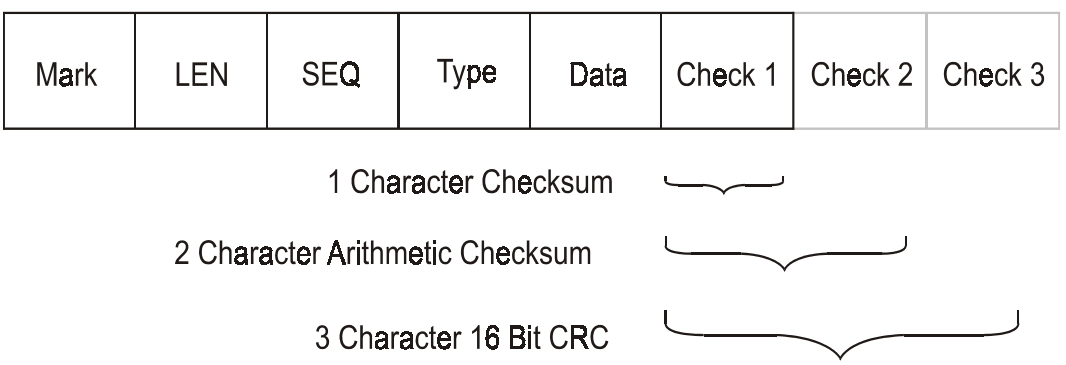


Figure 8.7
Kermit packet

Where:

Mark	This is the start of packet signature byte, SOH. This is the only canonical control bit permitted in the packet.
LEN	Number of bytes in the packet following this field.
SEQ	Characterized packet sequence number, Modulo-64.
Type	A single literal ASCII character identifying the type of packet (e.g. 'D' for data).
Data	The actual packet type determines the contents of this field.
Check 1	Packet check value.
Check 2	
Check 3	

Note: Kermit converts 'high risk' characters to printable characters before transmission. Control characters are thus moved to printable form before transmission and then transformed back after reception. High-risk characters are those that the computer system may elect to modify or throw away because of its system philosophy.

Description	XMODEM	YMODEM	ZMODEM	KERMIT
Types of File Transfer				
Single File	Yes	Yes	Yes	Yes
Batch of Files	No	Yes	Yes	Yes
Text/Data/Binary	Yes	Yes	Yes	Yes
Number of Data Bits	8	8	8	7 or 8
Data Compression	No	No	Yes	Yes
Error Checking	Checksum	Checksum/ CRC-CCITT	CRC-CCITT/ CRC-32	Checksum/ CRC-CCITT
Error Checking Response	Single char.	Single char.	Single char.	Error checked pkt
Data Packet Size (bytes)	128	1024	1024	0-95
File Attribute Transfer				
File Size/Time/ Date Stamp	No	Yes	Yes	Yes
Sliding Window Support	Single pkt	Single pkt	1 - more pkts	1 - more pkts
Negotiated Parameters				
Packet Size	No	Yes	Yes	Yes
Packets in Window	No	No	Yes	Yes
Error Check Type	No	Yes	Yes	Yes
Data Compression	No	No	Yes	Yes
XON/XOFF Control	No	No	Yes	Yes

Table 8.4
File transfer protocols comparison